

Mizar Evolution: **なぜ今、一階述語論理なのか**
自動証明・読める数学・検証できる計算を再接続する
TPP 2026、**理化学研究所 AIP 東京オフィス**、2026 年 11 月

Mizar Evo project

TPP 2026、**理化学研究所 AIP 東京オフィス**、2026 年 11 月 16–17 日

レベル	意味	表示
事実	既存システム、公開ベンチマーク、 Mizar Evo の仕様と main branch	無印
研究仮説	Mizar Evo が検証するために作ら れている主張	本文中に明記
将来構想	日程も設計もまだ決まっていない 目標	本文中に明記

コード例のラベルは Białystok 資料と同じ: exact MML excerpt、specification example、sketch。

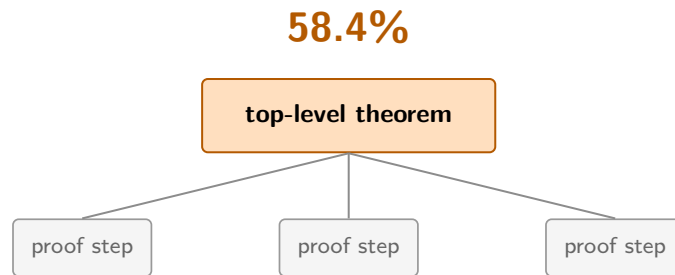
- 仕様と実装は別物です。終盤の一枚でその境界を示します。

1.1 - 似て見える二つの数字

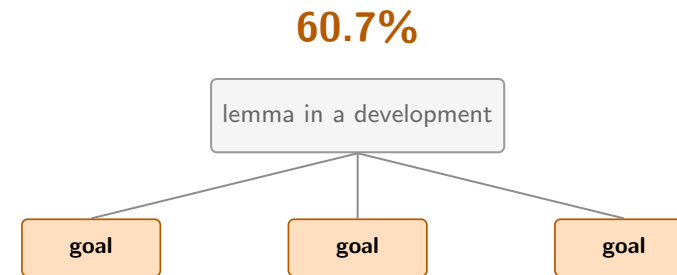
	MizAR 60 (ITP 2023)	Sledgehammer on AFP (CICM 2015)
見出しの数字	58.4% を証明	60.7% を証明
数える単位	MML 1147 の top-level theorem と lemma (57,897 件)	AFP 開発内部の proof goal (6,934 件)
前提	ライブラリ全体から学習的に選択	開発内から MePo フィルタで選択
時間予算	ポートフォリオ合計 420 CPU 秒	各 prover 30 秒、4 prover
結果の扱い	ATP 証明が見つかった (hammering mode)	union を oracle として信頼。一行再構成は各 prover 約 50%

- 二つのハンマー、二つの見出しの数字。ほとんど同じに見えます。
- 一つのベンチマークとして比べてはいけません。単位も、前提も、時間予算も違います。

1.2 - それぞれの数字は何を数えているか



Mizar 60 (ITP 2023) counts *whole top-level theorems and lemmas* 57,897 in MML 1147; premises from the whole library; 420 CPU s portfolio

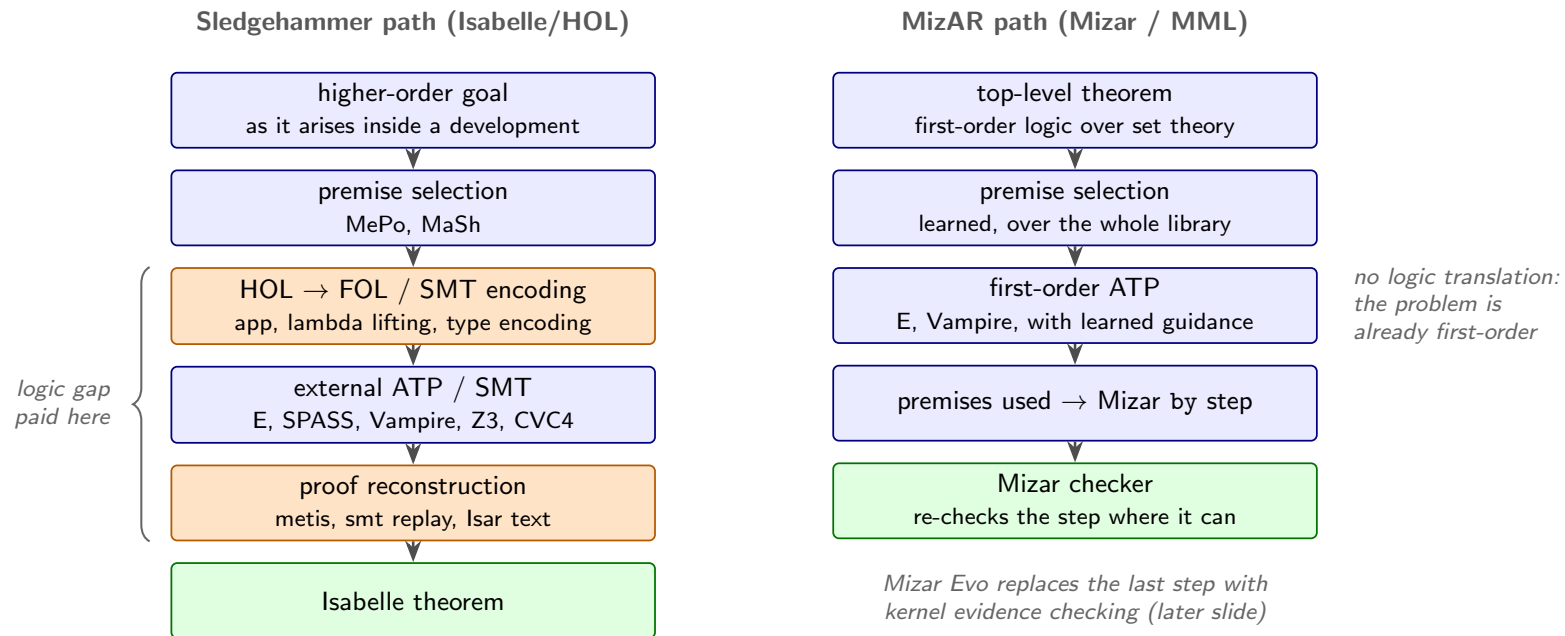


Sledgehammer on the AFP (CICM 2015) counts *proof goals as they arise* 6,934 goals from 128 theories; 30 s per prover; union trusted as oracle

Same-looking percentages. Different units, different premise regimes, different budgets.

- MizAR の問い: この定理を丸ごと、著者の助けなしに、ライブラリから機械が証明できるか。
- AFP 研究の問い: 現れた場所でこの goal を機械が閉じられるか。多くは人がすでに書いた証明の内側にあります。
- どちらも正当な問いです。しかし同じ問いではありません。

1.3 - 一階 prover へ至る二つの経路



- Sledgehammer は高階の goal を一階論理か SMT の問題に翻訳します。そのあと証明を Isabelle の内側に組み立て直します。
- MizAR は、最初から一階である問題を prover に渡します。検査系と ATP 問題の距離が短いのです。

1.4 - 問い

測り方の違いなのか、設計の帰結なのか

- **研究仮説:** 一階のライブラリでは、検査系自身の論理と ATP 問題の距離が短い。高階の処理系はその距離を翻訳と再構成で渡る必要があり、そこに代償がある。
- **事実:** Sledgehammer は、高階の処理系が一階 ATP を非常にうまく使えることを示しました。
- **研究仮説:** 「高階プラス翻訳」が ATP を使う最良の設計かどうかは、まだ分かりません。比較対象になる現代的な一階 ITP がほとんど無いからです。
- **Mizar Evo は、現代的な検査系・ライブラリ・prover でこの仮説を試すために作っています。**

2.1 - 高階論理における関数

HOL の文の概形 **sketch** (Isabelle/HOL 風の記法)

```
f :: 'a => 'b      x :: 'a
```

```
P (f x)
```

- HOL では、関数型と関数適用が論理そのものの一部です。
- 関数を引数にする、関数を返す関数、部分適用、ラムダ。すべて直接書けます。
- 数学を書く人にとって、これはとても便利です。
- しかし E や Vampire は一階の prover です。高階の構造は、prover に見せる前に符号化しなければなりません。

2.2 - 代償は ATP との境界で現れる

符号化の手順の概形 **sketch**

<code>F X</code>	<code>-> app(F, X)</code>
<code>(%x. t) ...</code>	<code>-> fresh constant + defining axioms (lambda lifting)</code>
<code>polymorphic types</code>	<code>-> type guards or type tags</code>
<code>Boolean-valued terms</code>	<code>-> extra encoding</code>

- 変数である関数の適用は、明示的な `app` 記号になります。ラムダは外に持ち上げるか、コンビネータにします。型は `guard` か `tag` になります。
- `prover` が成功したら、証明を `Isabelle` に戻さなければなりません。 `metis` の呼び出し、 `smt` の `replay`、生成した `Isar` テキスト。この工程が再構成 (`reconstruction`) です。
- これは優れた工学的成果です。同時に、一階の処理系には不要な層でもあります。
- `HOL` は便利さを先に受け取り、ATP との境界で支払います。

2.3 - 集合論における関数: Mizar の基礎

関数適用は定義された集合論的關係 exact MML excerpt

```
func f.x -> set means  
:Def2:  
[x,it] in f if x in dom f otherwise it = {};
```

型付き関数は部分関数の上の soft type exact MML excerpt

```
definition  
  let X,Y;  
  mode Function of X,Y is quasi_total PartFunc of X,Y;  
end;
```

- ここでは関数は普通の一階の対象、つまり対の集合です。適用は論理に組み込まれているのではなく、定義されています。
- 基礎の論理に高階である必要はどこにもありません。
- 代償: 素朴に集合論を書くと、定義域、グラフ、関数性、所属関係がすべて本文に現れます。

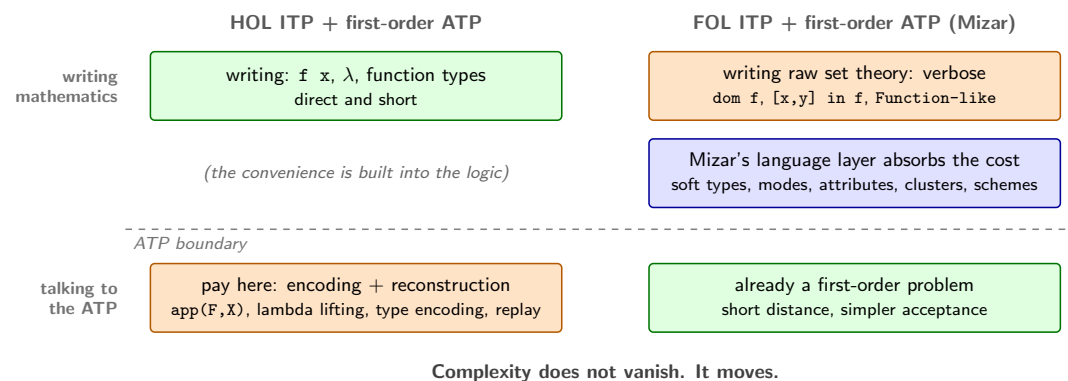
2.4 - Mizar の 50 年の答え: 細部を言語に隠す

著者が実際に書くもの **sketch** (現行 Mizar と Mizar Evo で有効)

```
let X, Y be set;  
let f be Function of X, Y;  
let x be Element of X;  
... f.x ...
```

- 同じ集合論的な関数です。しかし著者が書くのは `Function of X,Y` と `f.x` であって、対や定義域ではありません。
- `soft type`、`mode`、`attribute`、`registration`、`scheme`、宣言的証明は、同じものの書きやすい別表記ではありません。一階の集合論を、読める数学へ持ち上げる言語設計です。
- Mizar はこの方向に 50 年歩いてきました。Mizar Evo は歩き続けます。

2.5 - どこで払うか



***HOL* と *FOL* は複雑さを消してはいない。
置く場所が違うだけである。**

- **HOL は ATP との境界で払う。FOL はキーボードの前で払い、Mizar の言語がその支払いを引き受ける。**
- **Mizar Evo の選択: 基礎の論理は一階のまま保ち、人間向けの複雑さは言語に隠す。**

2.6 - トレードオフを一枚の表で

deep dive

	HOL ITP + ATP	FOL ITP + ATP
記述	高階の機能、短い本文	素朴に書くと長い
関数	基本的な高階の対象	一階の集合論的对象
ATP 接続	符号化が必要	距離が短い
再構成	論理のギャップを戻す必要	原理的には単純
言語が提供すべきもの	HOL の抽象	一階の細部を隠す抽象

- この表は解釈であって、測定ではありません。測定は Mizar Evo が 2027 年に出すべきものです。

3.1 - 論理は保ち、言語を現代化する

**数学の層は保つ。
その下と周りの道具を作り直す。**

- Mizar Evo は、一階論理と Tarski-Grothendieck 集合論を基礎の論理として保ちます。
- soft type、mode、attribute、registration、structure、**宣言的証明も保ちます。**
- 隠れていた選択を明示し、generic な仕組みを統一し、自動化を追跡できるようにし、全体を現代的なコンパイラ構成に載せます。
- 続く数枚で「言語を現代化する」の中身を示します。template、algorithm、信頼境界、基盤です。

3.2 - Template: 高階論理なしの generic な数学

template はパラメータ付きの definition ブロック specification example

```
definition
  let T be type;
  struct MagmaStr[T] where
    field carrier -> T;
    field binop -> BinOp of T;
  end;
end;
```

- パラメータは型、値、述語、関数子を取れます。古典的な Mizar の scheme は、述語パラメータ付きの定理として同じ仕組みに入ります。
- template は論理に無制限の二階量化を追加しません。インスタンス化はそれぞれ検査され、一階の証明義務を生みます。
- generic な数学は表層言語に住み、基礎の論理には住みません。

述語パラメータ付き定理としての帰納法 specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

- 述語パラメータは、代入する述語ごとに一つ、一階の定理の族を与えます。
- インスタンス化は明示的です: `defpred` のあとに `by NatInduction[P], Base, Step`.
- 関数子パラメータは `schema` レベルの記号であって、集合ではありません。これが論理を一階に保ちます。
- 詳細: Białystok 資料 Story 6 (`PermProduct[T]`、`qua` によるビュー)、および本資料 Backup 4。

3.4 - Algorithm は第二の柱

- Mizar Evo の algorithm は、高階関数の代わりではありません。別の必要、つまりアルゴリズムについての推論と、検査できる計算に答えるものです。
- 歴史的な傾向であって必然ではありません: 一階論理には完全な推論系があり、その周りに resolution から superposition、saturation へと自動探索の文化が育ちました。
- LCF と HOL の系統には、プログラムできる証明構成の文化、tactic と tactical が育ちました。
- Mizar は宣言的証明と、処理系に組み込まれた自動化を使ってきました。ユーザがプログラムできる tactic 言語は持ちませんでした。
- Mizar Evo の algorithm は、上に載せた tactic 言語ではありません。契約を持つ手続きであり、処理系がそれを検証します。

3.5 - Algorithm: 契約、証明、計算

契約付きのユークリッドの互除法 specification example

```
terminating algorithm euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a;  var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y);  decreasing y;
    const r := x mod y;  x := y;  y := r;
  end;
  return x;
end;
```

- 契約、不変条件、停止性の測度は一階の証明義務になり、定理と同じように検査されます。
- 役割は二つ。検証された自動化手続きと、by computation で実行する検査済みの計算。状態: 仕様化済み。MVM による実行とコード抽出は後の作業です。

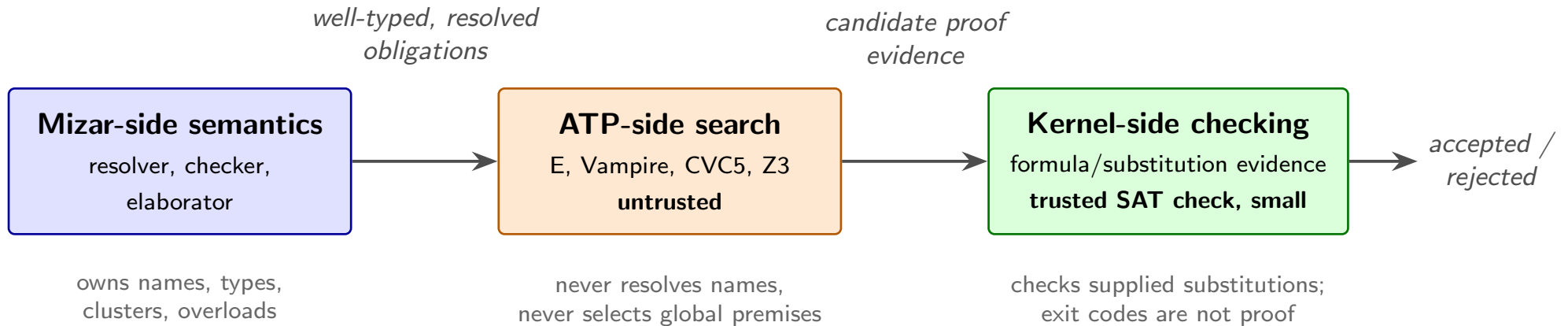
将来構想であり、現在の能力ではない:

- 整数論・組合せ論のアルゴリズム、記号計算、最適化手続き。
- さらに先: 暗号アルゴリズムとプロトコル、量子アルゴリズムと古典・量子ハイブリッド。

すべてに共通する一つの流れ:

1. 手続きを書く。
 2. 契約を述べる。
 3. 不変条件と停止性を与える。
 4. 証明義務を生成する。
 5. ATP で証明し kernel で検査する。
 6. 具体的な入力で実行する。
 7. 将来はコードを抽出する。
- algorithm は tactic と応用アルゴリズムの間のギャップを小さくします。上の対象は将来構想です。

3.7 - 探索は外、信頼は内



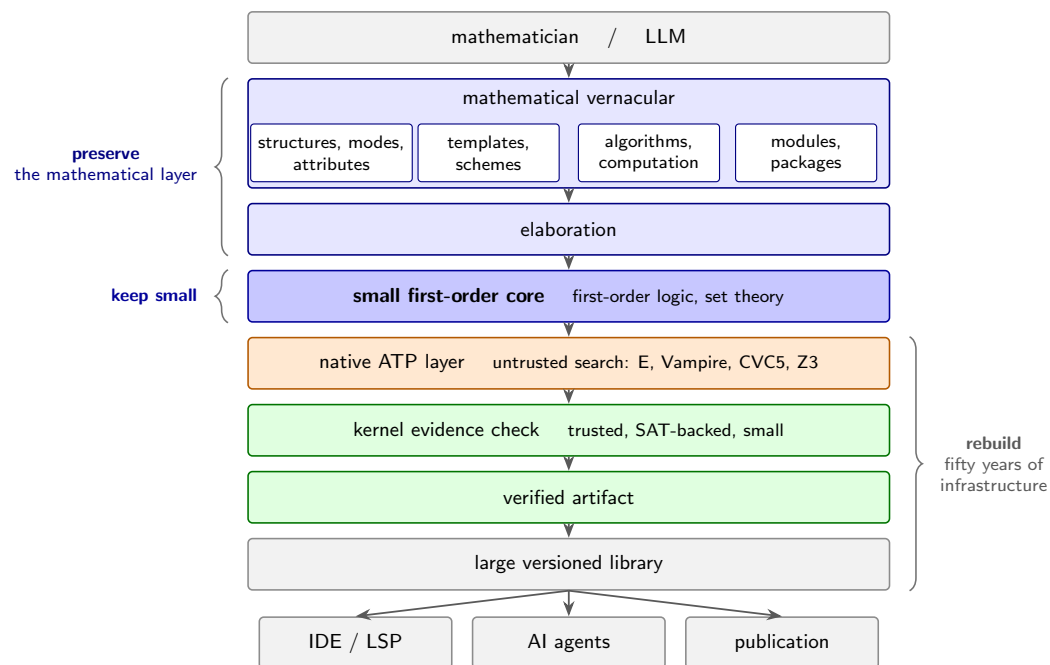
- 一階 ATP は強力な探索器です。信頼できる検査器ではありません。
- Mizar 側が名前、型、cluster、オーバーロードを受け持ちます。prover は探索を受け持ちます。kernel は受理を受け持ち、渡された論理式と代入を、小さく信頼できる SAT 検査で確かめます。
- prover の終了コードは証明ではありません。だから一階の自動化を設計原理にしても、信頼基盤は大きくなりません。

3.8 - そして Mizar 自身も 50 年経っている

MML 50 年で見えたこと	Mizar Evo
article が依存の単位 グローバルな名前管理 配布とバージョン管理が弱い フルビルド ATP は外付け、自動化が見えにくい IDE 連携と機械可読な入出力が弱い	明示的 import を持つモジュール namespace、完全修飾名 package、SemVer、lock file 依存指紋による差分ビルド 第一級の ATP パイプライン、解決トレース、 kernel evidence LSP、構造化診断、エージェント向けインタフェース

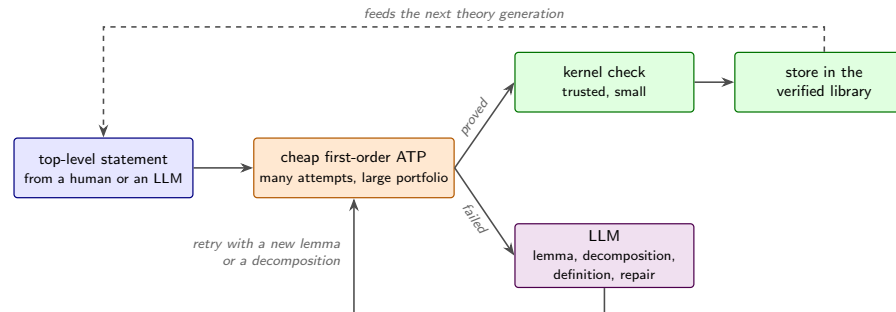
- Mizar の数学的な考え方を保つことと、1970 年代のソフトウェア構成を保つことは、別のことです。
- もちろん開発基盤全体も現代化します。詳細は Białystok 資料にあります。ここでは一枚の表で十分です。

4.1 - 全体像



上に豊かな数学。下に小さな一階論理。
周りに現代的な基盤。

4.2 - LLM が考え、ATP が証明し、Mizar Evo が記憶し検証する



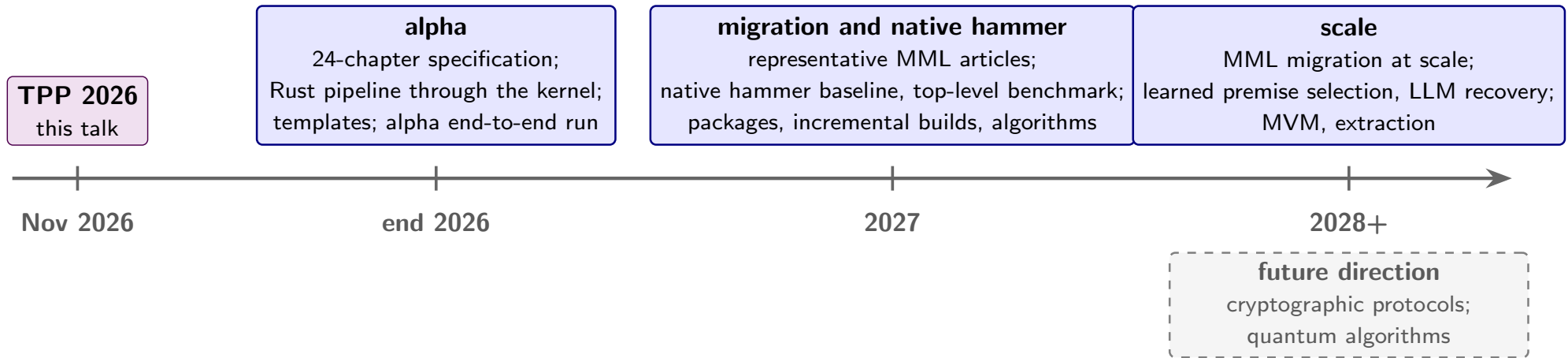
LLM thinks. ATP proves. Mizar Evo remembers and verifies.
division of labor: design intent — cost-effectiveness of the cascade: research hypothesis

- **LLM: 理論、定義、方針、補題、失敗からの回復。ATP: 安価で反復できる一階の探索。Mizar Evo: 表現、検証済みライブラリ、信頼できる検査、各事実の来歴。**
- **研究仮説: 機械が大量の数学を生成する時代には、ATP による安価な検査の価値が上がる。このループの費用と効果はまだ測られていません。**
- **現在の仕様: ATP が受け取るのは引用された前提と局所仮定だけです。ライブラリ全体を使うハンマーは 2027 年の研究です。**

4.3 - プロジェクトの現在地 (2026 年 10 月)

- 仕様: 24 章と付録。英語が正典。
- main branch に実装済み: Rust フロントエンド (字句解析、構文解析、構文木)。alpha コーパス上の名前解決と型検査。証明義務の生成と決定的な discharge。ATP 問題の符号化と候補 evidence。SAT に基づく kernel の evidence 検査。キャッシュ、指紋、ビルドスケジューリングの各マイルストーン。
- 進行中: ソースから検証済み成果物までの end-to-end 統合。LSP サーバ。ドキュメント生成。
- 後の作業: MVM の実行、コード抽出、ライブラリ全体の前提選択、MML の移行。
- この講演では、外部 prover を使った end-to-end の結果は主張しません。

5.1 - ロードマップ



- 2026 年: 仕様、kernel までの Rust パイプライン、template 処理、alpha の end-to-end 実行を仕上げる。
- 2027 年: 代表的な MML article を移行し、native hammer のベースラインを作り、MizAR と同じ単位、top-level theorem でベンチマークする。
- 2028 年以降: 移行の拡大、学習ベースの前提選択、LLM による失敗回復、MVM と抽出。暗号と量子は将来構想のまま。

5.2 - 二つの経路に戻る

問いは、高階の処理系が一階 ATP を使えるか、ではない。
明らかに使える。

問いは、一階の自動推論を最初から設計原理にしたとき、
何が可能になるか、である。

Mizar Evo の答え、六つ:

1. 一階論理と集合論を基礎の論理として残す。
2. Mizar の言語設計で一階の細部を隠す。
3. template で generic な数学を広げる。
4. algorithm で検査できる計算を加える。
5. 50 年分のソフトウェア基盤を作り直す。
6. LLM と ATP を、それぞれが強い場所で組み合わせる。

5.3 - おわりに

*Keep the foundation small.
Keep the mathematics readable.
Modernize everything else.*

- 基盤は小さく。数学は読めるように。それ以外はすべて現代化する。
- ありがとうございました。特に、2027 年のベンチマークを「同じものを同じ単位で比べる」設計にする方法について、ご意見をいただければ幸いです。

Backup 1. MizAR 60 の詳細

Source: Jakubův, Chvalovský, Goertzel, Kaliszyk, Olšák, Piotrowski, Schulz, Suda, Urban, MizAR 60 for Mizar 50, ITP 2023.

- **データ**: MPTP で出力した MML 1147、無名の top-level lemma を含む 57,897 件の定理。MizAR 40 の評価と同じ版なので比較できる。
- top-level lemma の 58.4% を large-theory (hammering) mode で、ユーザの助けなしに、CPU 時間 420 秒に制限したポートフォリオで証明 (MizAR 40 は約 40.6%)。
- 人または機械がライブラリから前提を選べる条件では 75% 超 (MizAR 40 は 56%)。
- 最強の単一手法: hammering mode で 30 秒 40%。人間の前提ありで 120 秒 60%。
- 転移: 最強手法は MML 1382 の新規 242 article、13,370 定理でも動く。
- 手法: ENIGMA と Deepire で誘導した E と Vampire、学習ベースの前提選択、数百万の ATP 証明で学習するループ。

Backup 2. Sledgehammer 評価の詳細

Prover	One-line / + Isar / + Oracle (6,934 goal に対する %)
E	49.7 / 51.4 / 52.5
SPASS	49.4 / 50.5 / 52.0
Vampire	49.5 / 51.0 / 51.8
Z3	49.6 / 50.0 / 53.7

- **設定: ランダムに選んだ AFP の 128 theory、各 100 goal まで。Isabelle2014。MePo フィルタ。各 prover 30 秒をスライスに分割。再構成は 2 秒以内に成功する必要。**
- **組み合わせて oracle として信頼すると、goal の 60.7% を証明。**
- **Judgement Day (2010) : 7 theory の 1,240 subgoal、E・SPASS・Vampire を 30 秒で 46%。2015 年の予備評価では 6 prover で 75%。**
- **theory ごとの成功率は 10% から 100% までばらつく。Sledgehammer は Judgement Day に合わせて調整されてきた。**

Backup 3. HOL から FOL への符号化と再構成

- **関数適用**: 変数である関数を引数に適用すると `app(F, X)` になる。定数の適用はカーリー化のままか平坦化。
- **ラムダ抽象**: `lambda lifting` は定義式付きの新しい定数を導入する。コンビネータ変換が代替。
- **型**: 多相な HOL の型は `guard`、`tag`、または単相化で符号化する。選択は健全性、完全性、`prover` の性能に影響する。
- **真偽値**: 論理式の中の真偽値を取る項には別の符号化が必要。
- **再構成**: 使われた補題による `metis`、SMT 証明の `smt replay`、生成した `Isar` テキスト。評価では再構成の失敗を別に数える。

Source: Meng and Paulson 2008; Blanchette, Böhme, Popescu, Smallbone 2016; Blanchette, Kaliszyk, Paulson, Urban 2016; Schurr, Fleury, Desharnais 2021.

Backup 4. Template のインスタンス化: 一つの証明、多くのビュー

有界な型パラメータと generic な定理 **specification example** (証明は省略)

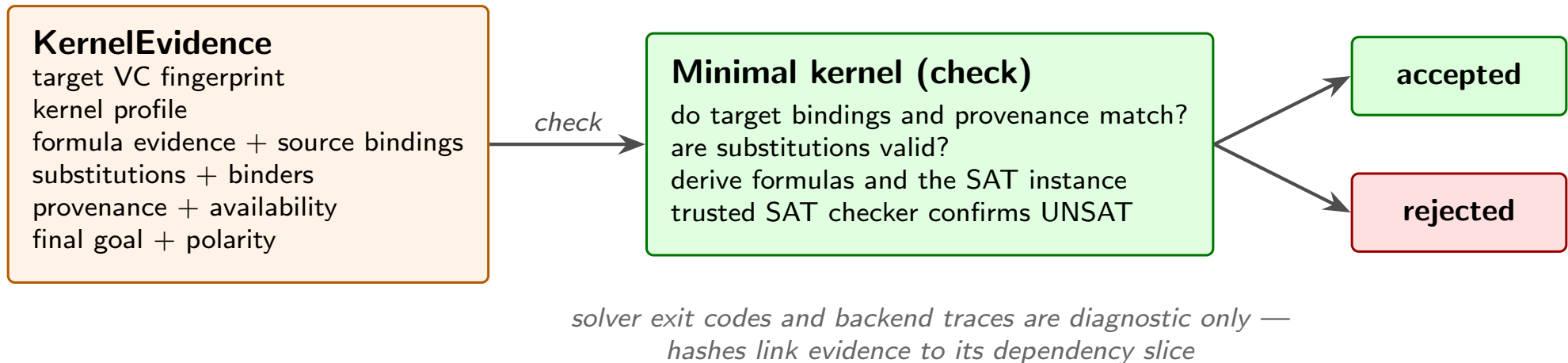
```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

インスタンス化 **specification example** (必要な registration がある前提)

```
PermProduct[commutative associative unital AddMagma]
PermProduct[commutative associative unital MulMagma]
let R be commutative Ring; PermProduct[R qua AddMagma] :: additive view
```

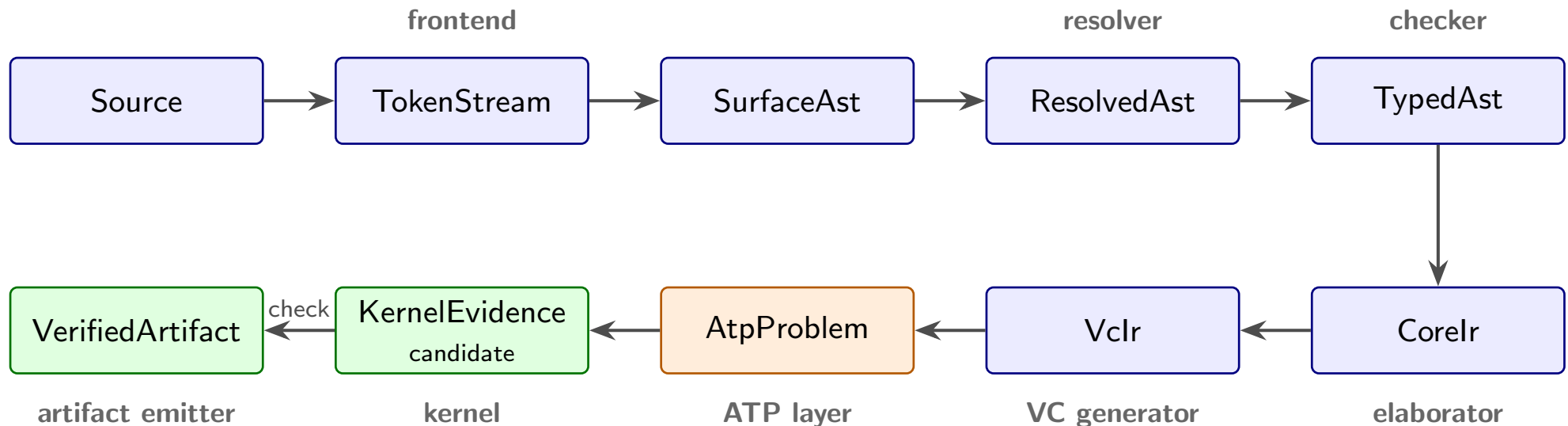
- 環は二つの経路で Magma に届く。qua がビューを選び、ビューが記法も決める。

Backup 5. Kernel evidence と信頼できる SAT 検査



- evidence は元の論理式、明示的な代入、来歴、対象と goal の束縛を持つ。kernel はそれらを検査し、決定的な SAT 問題を作り、信頼できるプロセス内 SAT 検査器に UNSAT を要求する。
- バックエンドの証明トレース、SMT の証明オブジェクト、ログ、終了コードは診断用のみ。

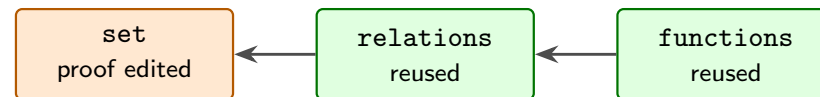
Backup 6. ATP の中心経路



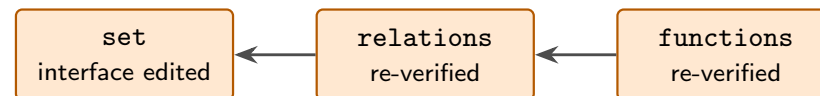
- 決定的な discharge のあとも開いている義務だけが ATP に行く。前段の discharge にも再生できる evidence が要る。
- 各境界は、誰が事実を所有し、どの成果物が記録し、変更後に何を再検査するかを定める。

Backup 7. 差分検証

theorem proof edit:
public statement + accepted status unchanged — importers reuse



interface change:
the dependency cone re-verifies



reuse requires all relevant keys: source/dependency slices, package/lockfile, toolchain/schema, registration/cluster, verifier policy, evidence/witnesses; missing data is a cache miss — a clean build must reproduce every acceptance

- 証明本体の編集は、公開された主張と受理状態が変わらなければ、importer を再ビルドしない。インタフェースの変更は依存コーンを再検証する。
- 関係するキャッシュキーはすべて一致する必要がある。データが無ければキャッシュミス。キャッシュ再利用は証明の権威ではない。クリーンビルドはすべての受理を再現しなければならない。

Backup 8. Package manifest

Mizar Evo **specification example**

```
[package]
name      = "algebra"
version   = "2.3.1"
edition   = "2025"

[dependencies]
mml_core  = "^1.0"
topology  = { version = "^0.9", features = ["metric"] }
```

- 再現可能なビルドには、固定したソース、lockfile、ツールチェーン、検査器の設定が要る。決定的な ATP evidence も含む。
- バージョン付きの再利用が article 集合間の手作業コピーを置き換える。集約モジュールは一つの import で一分野をまとめて公開できる。

Source: doc/spec/en/23.package_management_and_build_system.md; Białystok 資料 Story 1。

Backup 9. 45 分版での追加

物語は変えない。次の順で具体例を深める:

1. **一つの goal で HOL から FOL への関数の符号化を追う** (app、部分適用、lambda lifting)。
2. Mizar **の関数表現**: Function of X, Y 、 $f.x$ 、**集合論的对象としての** soft type。
3. Template: **加法と乗法のビューを持つ** PermProduct[T] (Backup 4)。
4. Algorithm: **ユークリッドの互除法**、by computation、**関数子への昇格**。
5. Structure **とビュー**: AddMagma、MulMagma、Magma (Białystok Story 2)。
6. **現代的な基盤**: mizar.pkg、namespace、**指紋グラフ** (Backup 7-8)。
7. **信頼境界**: ATP 探索、KernelEvidence、**検査された受理** (Backup 5)。

Backup 10. 出典と帰属

この講演で使う MML の正確な引用:

目的	出典	行	使用箇所
関数適用 <code>f.x</code>	<code>funct_1.miz</code>	138-140	Frame 2.3
<code>soft type</code> としての <code>Function of X,Y</code>	<code>funct_2.miz</code>	87-90	Frame 2.3

Source URLs:

- `FUNCT_1`: https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz
- `FUNCT_2`: https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz

帰属についての注記:

- MML のテキストは GPL-3.0-or-later / CC-BY-SA-3.0-or-later。article 名、URL、行番号を発表者ノートに残す。
- ベンチマークの数字は Backup 1-2 と `references.bib` を参照。最終版の前に書誌情報を出版社で確認する。